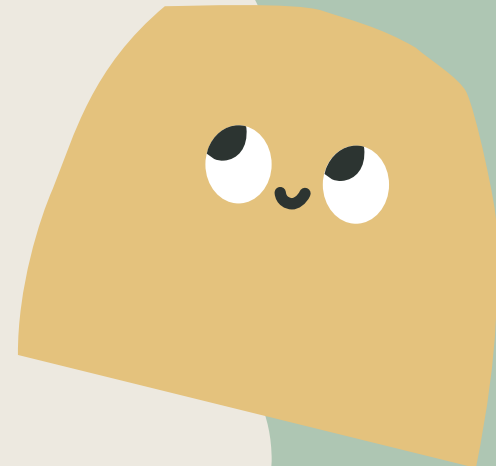
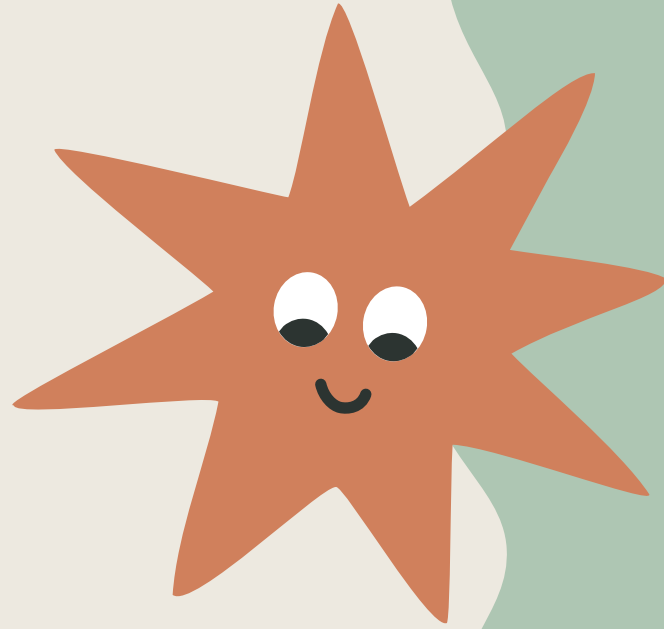


Asymptotic Analysis & Big O Notation

Lecture 4



Learning Outcomes (Lecture 4)

- At the end of this lecture, students will be able to:
- Explain the purpose of asymptotic analysis.
 - Differentiate between Big O, Big Ω , and Big Θ notations.
 - Explain Best, Average, and Worst Case.
 - Calculate the time complexity of simple C++ programs.
 - Calculate the time complexity of nested loops.
 - Analyze logarithmic loops.
 - Compare different algorithms using asymptotic notations.

Motivation

➤ Suppose there are two students

➤ Student A write

```
for(int i=0;i<n;i++)
```

➤ Student B write

```
for(int i=0;i<n*n;i++)
```

➤ Without running the program,

- Which algorithm is likely to perform better?
- How can we prove it?

➤ Today's lecture answer this question

What is Asymptotic Analysis?

- Asymptotic Analysis is a mathematical technique used to describe how an algorithm behaves as the input size becomes very large — approaching infinity (∞).
- Why ignore small inputs?
 - Small inputs are fast on almost any algorithm. Modern CPUs execute billions of instructions per second meaning the execution time for small inputs is usually measured in fractions of a microsecond.
 - Real-world systems are defined by “Worst-Case/Large Data Loads”. In software, our app will never crash or run slow when testing with 3 sample users. Our app **will** crash when 100,000 users log in at the same time on Black Friday.

Best, Average and Worst Case

- Suppose we've following array

10 20 30 40 50

- Searching 10, found immediately, **Best Case**, minimum work
- Searching 30, found in middle, **Average Case**, average work
- Searching 50, need all comparisons, **Worst Case**, maximum work

Core Asymptotic Notations

1. Big O Notation (O) — Upper Bound

- Big O describes the upper bound (maximum growth, worst-case) of an algorithm
- This is the primary notation used in DSA interviews, exams, and industry practice because software systems must guarantee performance under stress

2. Big Omega Notation (Ω) — Lower Bound

- Big Ω represents the minimum amount of work an algorithm must perform (best-case lower bound)

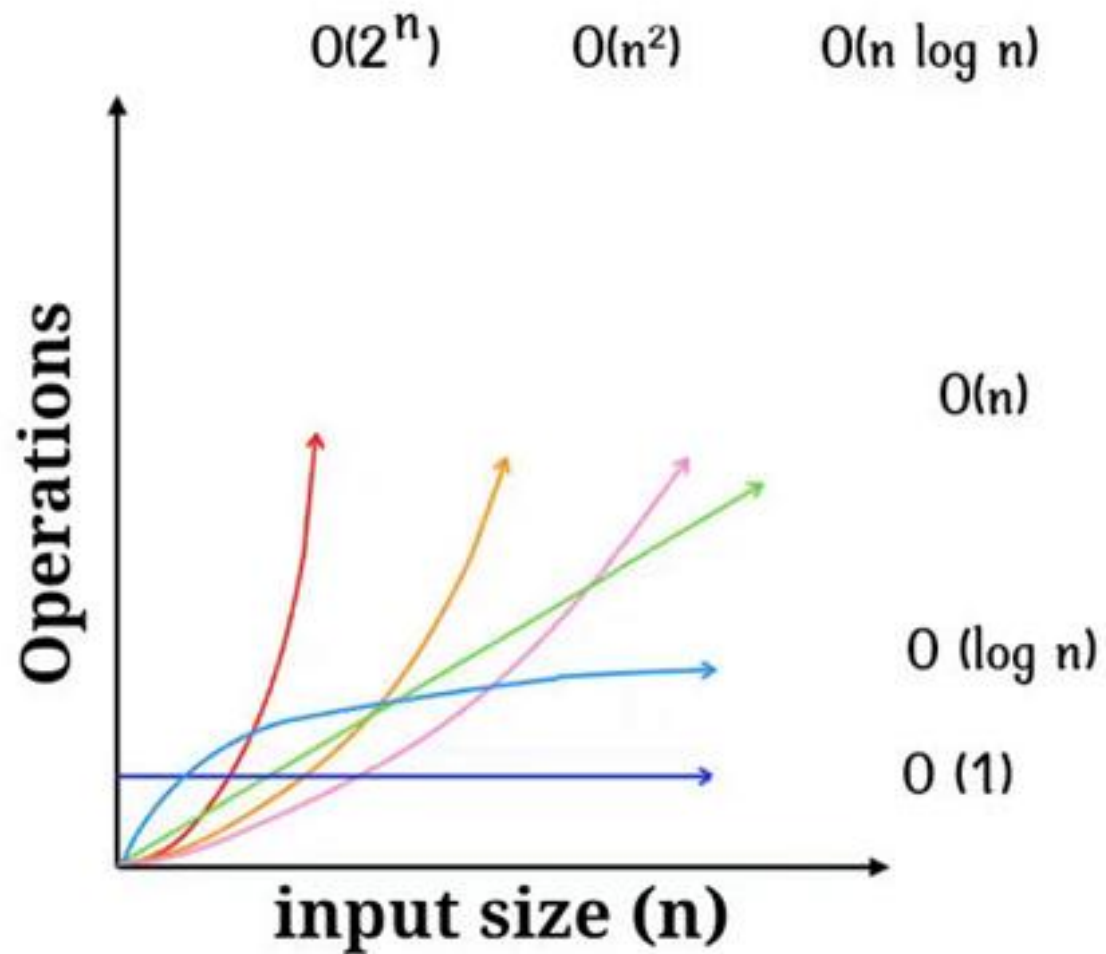
3. Big Theta Notation (Θ) — Tight Bound

- Big Θ represents the exact growth rate when the upper and lower bounds are the same

Common Growth Rate Hierarchy

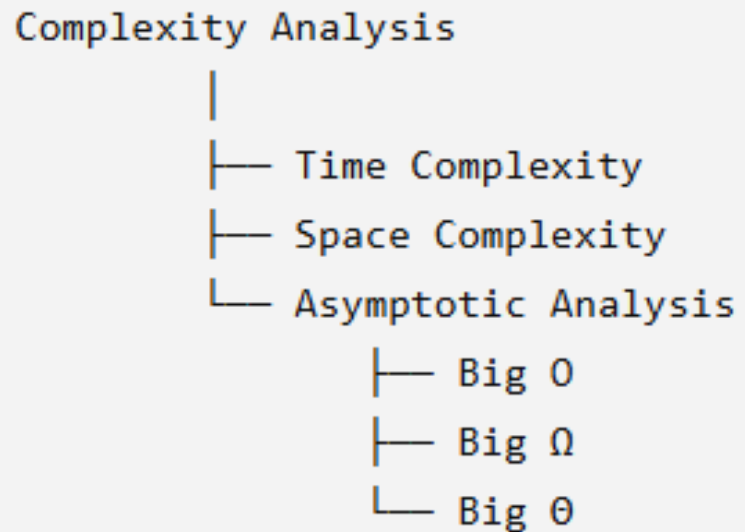
Notation	Name	Practical Example
$\mathcal{O}(1)$	Constant	Accessing an array element by index
$\mathcal{O}(\log n)$	Logarithmic	Binary Search
$\mathcal{O}(n)$	Linear	Iterating through an array
$\mathcal{O}(n \log n)$	Linearithmic	Merge Sort, Quick Sort (average)
$\mathcal{O}(n^2)$	Quadratic	Nested loops (e.g., Bubble Sort)
$\mathcal{O}(2^n)$	Exponential	Recursive Fibonacci calculation
$\mathcal{O}(n!)$	Factorial	Traveling Salesperson Problem (Brute Force)

Common Growth Rates



Complexity Analysis vs Asymptotic Analysis

- **Complexity Analysis** is the **broader concept** of evaluating an algorithm's efficiency (time and space)
- **Asymptotic Analysis** is a **specific technique** within complexity analysis that describes how an algorithm behaves as the input size becomes very large using **Big O**, **Big Ω** , and **Big Θ**



Example 1 – Single Statement

```
int a = 10;
```

➤ Step 1

- Operations = 1

➤ Step 2

- $T(n) = 1$
- $T(n)$ = Total number of operations (or running time) performed by an algorithm as a function of the input size n

➤ Step 3

- Big O
- $O(1)$

Example 2 – Multiple Statements

```
int a=10;  
int b=20;  
int c=a+b;
```

➤ Step 1

- Operations = $1 + 1 + 1 = 3$

➤ Step 2

- $T(n) = 3$

➤ Step 3

- $O(1)$

Prepared By: Ehtisham Rasheed

Example 3 – Loop

➤ Operations

- `int i = 0` executes **1** times
- `i < n` executes **n + 1** times
- `i++` executes **n** times
- `cout << i` executes **n** times

➤ Total Time

- $T(n) = 1 + n + 1 + n + n$
- $T(n) = 3n + 2$

➤ Big O

- $O(n)$

```
for(int i=0;i<n;i++)  
{  
    cout<<i;  
}
```

Example 4 – Loop

➤ Operations

- 1, $n + 1$, n , n , n

➤ Total Time

- $T(n) = 1 + n + 1 + n + n + n$
- $T(n) = 4n + 2$

➤ Big O

- $O(n)$

```
for(int i=0;i<n;i++)  
{  
    x++;  
    y++;  
}
```

Example 5 – Loop

➤ Operations – Loop 1

- 1, n + 1, n

➤ Operations – Loop 2

- 1, n + 1, n

➤ Total Time

- $T(n) = 1 + n + 1 + n + 1 + n + 1 + n$
- $T(n) = 4n + 4$

➤ Big O

- $O(n)$

```
for(int i=0;i<n;i++)  
{  
  
}  
  
for(int j=0;j<n;j++)  
{  
  
}
```

Example 6 – Loop

```
int sum( int n )
```

```
{
```

```
    int partialSum;
```

```
    partialSum = 0;
```

```
    for( int i = 1; i <= n; ++i )
```

```
        partialSum += i * i * i;
```

```
    return partialSum;
```

```
}
```

1

$T(n) = 3n + 3$

1, n, n

Complexity = $O(n)$

n

1

Prep

Example 7 – Nested Loop

- Outer loop runs **n** times
- Inner loop runs **n** times
- Multiply $n \times n$
 - $T(n) = n^2$
- Big O
 - $O(n^2)$

```
for(int i=0;i<n;i++)  
{  
    for(int j=0;j<n;j++)  
    {  
    }  
}
```


Example 8 – Nested Loop

- Outer loop runs **n** times
- Inner loop runs **m** times
- Multiply $n \times m$
 - $T(n) = nm$
- Big O
 - $O(nm)$

```
for(int i=0;i<n;i++)  
{  
    for(int j=0;j<m;j++)  
    {  
    }  
}
```